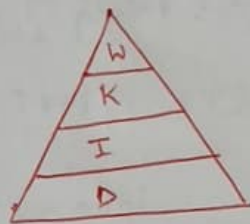


5 V's of Big Data

<u>Volume</u> ↓ Data at Rest	<u>Velocity</u> ↓ Data in Motion	<u>Variety</u> ↓ Data in Many forms	<u>Veracity</u> ↓ Data in Doubt	<u>Value</u> ↓ Data into money.
Another 5 →	Validity	Variability	Venue	Vocabulary Vagueness

DIKW [Data Information Knowledge Wisdom]



Storage as [NAS (Network Attached Storage)] and [SAN (Storage Area Network)]

Data is moved to compute nodes at compute time.

↓
When we have to deal with huge data, getting back the data to the processor becomes the bottleneck

↓
adding load to system → performance ↓, ↓, ↓

Solⁿ is — HADOOP → commodity Hardware.

Core Principles:

1. Scale-Out rather than Scale-Up
2. Bring code to data rather than data to code
3. Deal with failure — they are common
4. Abstract complexity of distributed & concurrent application

Hadoop Daemons — five separate daemons.

1. NameNode

2. Secondary NameNode

3. DataNode

4. JobTracker — Manages MapReduce jobs,

5. TaskTracker — Responsible for instantiating and monitoring individual Map & Reduce tasks.

Big Data

Midterm - 4 topics →
Hadoop
Spark
Rhadoop
Pig Latin.

Hadoop

- Hadoop is a popular open-source framework for processing large datasets in a distributed computing environment.
- It helps organizations process vast amounts of data (structured and unstructured) efficiently and cost-effectively.
- It works on the concept of distributed storage and distributed processing.

Key Components of Hadoop:

1. HDFS

This is the storage layer of Hadoop.

It stores large files across multiple machines, ensuring data redundancy, fault-tolerance, and scalability.

2. MapReduce

This is the processing layer of Hadoop.

It allows data processing to be divided into small tasks that can run in parallel across multiple machines.

HDFS (Hadoop Distributed file System)

- HDFS is the primary storage system of Hadoop.
- It is designed to store large files across many machines in a fault-tolerant manner.

Architecture:

1. Name Node: This is the master node that manages the metadata (information about where the data is stored) of the files in the system.
2. Data Nodes: These are the worker nodes where the actual data is stored. Each file is divided into blocks and distributed across these DataNodes.

Key features: Fault Tolerance, Scalability, Write once Read Many

- Block-Based Storage: files in HDFS are split into fixed-size blocks (typically 128 MB (or) 256 MB).

These blocks are distributed across multiple nodes in the cluster.

- Replication: Each block is replicated across multiple Datanodes (typically 3 times) to ensure fault tolerance. If a node fails, data can still be retrieved from another replica.

- High Throughput: HDFS is optimized for reading large datasets rather than for low-latency random reads.

Name Node (Master Node)	Datanode (Worker Node)	Secondary Name Node
- Manage Metadata (file name, location, replications).	- Store Actual Data.	- Acts as a backup for the NameNode.
- Doesnot store actual data.	- Respond to NameNode's request (read/write/replication)	- Periodically merges NameNode's logs and snapshots.
- Maintain a directory structure similar to traditional file system.		

Map Reduce

MapReduce is a programming model for processing large dataset in parallel across a distributed cluster of computers. It has two main phases:

• Map Phase:

The input data is divided into smaller chunks (usually one chunk per blocks in HDFS).

The Map function processes these chunks and outputs a set of Key-value pairs (intermediate results).

• Shuffle and Sort:

After the map phase, the systems sorts and groups the intermediate Key-value pairs by Key. These process is called shuffling.

• Reduce Phase:

The Reduce function takes these sorted and grouped Key-value pairs and processes them to produce the final output (aggregated results).

Input: Hello Hadoop Hadoop

Example of mapReduce Job: -
Map o/p: (Hello, 1) (Hadoop, 1) (Hadoop, 1)
Reduce phase: (Hello, 1) (Hadoop, 2)

If you want to count the number of occurrences of each word in a large text file:

1. The MapReduce function would break the text into words and emit a Key-value pair for each word (eg- ("word", 1))

2. The Reduce function would then sum the counts for each word to give the total occurrences.

Advantage of MapReduce:

1. Parallel Processing [Distributed tasks across multiple nodes]
2. Fault Tolerance [Automatic task re-execution in case of failure]
3. Optimized for large Data Processing

YARN (Yet Another Resource Negotiator)

YARN is the resource management layer at Hadoop. It was introduced in Hadoop 2.0 to improve the scalability and resource utilization of the Hadoop ecosystem.

YARN is responsible for managing and scheduling resources in a multi-tenant cluster.

Components of YARN:

1. **Resource Manager (RM):** The master daemon that manages the resources and schedules jobs on the cluster.
2. **Node Manager (NM):** The worker node daemon that is responsible for managing resources and monitoring the health of the node.
3. **Application Master (AM):** For each application/job, an Application Master is created to negotiate resources with the RM and manage the execution of the application on the cluster.

Hadoop Ecosystem

Hadoop is not just about HDFS and MapReduce. There is a large ecosystem of projects that enhance its capabilities for various use cases:

1. Hive
2. Pig
3. HBase
4. Sqoop
5. Flume
6. Oozie
7. Zookeeper

1 Hive : A data warehouse system build on the top of the Hadoop, which allows SQL like queries (HiveQL) to be run on large dataset.

2. Pig : A platform for analyzing large datasets using a high-level scripting language called Pig Latin.

3. HBase : A No SQL database that runs on the top of HDFS, designed for real-time read/write access to large datasets.

4. Sqoop : A tool for transferring bulk data between Hadoop and relational databases.

5. Flume : A service for collecting, aggregating, and moving large amount of log data into HDFS.

6. Oozie : A workflow scheduler for Hadoop that allows users to manage and schedule complex data processing pipelines.

7. ZooKeeper : A coordination service for distributed application ensuring synchronization and managing configuration.

for storage and Data Management —

[HBase
Hive
Pig]

Data Processing —

[Spark
Tez]

Data Ingestion —

[Sqoop
flume]

Workflow Management —

[Oozie]

Search & Indexing —

[Solr / Lucene]

Hadoop Cluster Setup

A Hadoop cluster consists of several machines (nodes) working together. There are two types of Nodes:

1. Master Node: This node coordinates the job scheduling and resource management (Resource Management, NameNode)
2. Worker Node: These nodes execute the tasks and store the actual data (DataNodes, NodeManagers)

Typically, a Hadoop cluster is divided into the following layers:

HDFS Layer: The storage layer (with NameNode and DataNodes).

Processing Layer: Where MapReduce jobs (or YARN resources) are executed (Node Manager, Application Master).

Hadoop Architecture Overview

Client Layer: The user or application that interacts with the Hadoop ecosystem.

Resource Management Layer: YARN, which schedules and manages resources.

Storage Layer: HDFS for data storage.

Processing Layer: MapReduce (or other processing frameworks like spark, Tez) for data processing.

Single Node vs Multi-Node Clusters

↓
All components (NameNode, DataNode, Resource Manager, NodeManager) run on single Machine

↓
Components are distributed across multiple machines for scalability and fault tolerance.

RACK awareness - Hadoop clusters are organized into racks (group of machines). Data Replication considers rack topology to minimize n/w congestion.

Fault Tolerance and Reliability in Hadoop

1. Data Replication : HDFS automatically replicates data blocks across different nodes to ensure fault tolerance. If a DataNode fails, another replica can be used.
2. Job Recovery : In case a node running a MapReduce task fails, YARN (or the Resource Manager) can reschedule the task to a different node.

Benefits of Hadoop

1. Scalability
2. Fault tolerance
3. Cost - Effective
4. Flexible

1.) Scalability - We can scale a Hadoop cluster by adding more machines, which allows for processing large amounts of data.

2.) Fault Tolerance - Data Replication ensure that if a machine fails, the data is still available.

3.) Cost - Effective : - Hadoop works on commodity hardware, making it cheaper to implement than traditional database solutions.

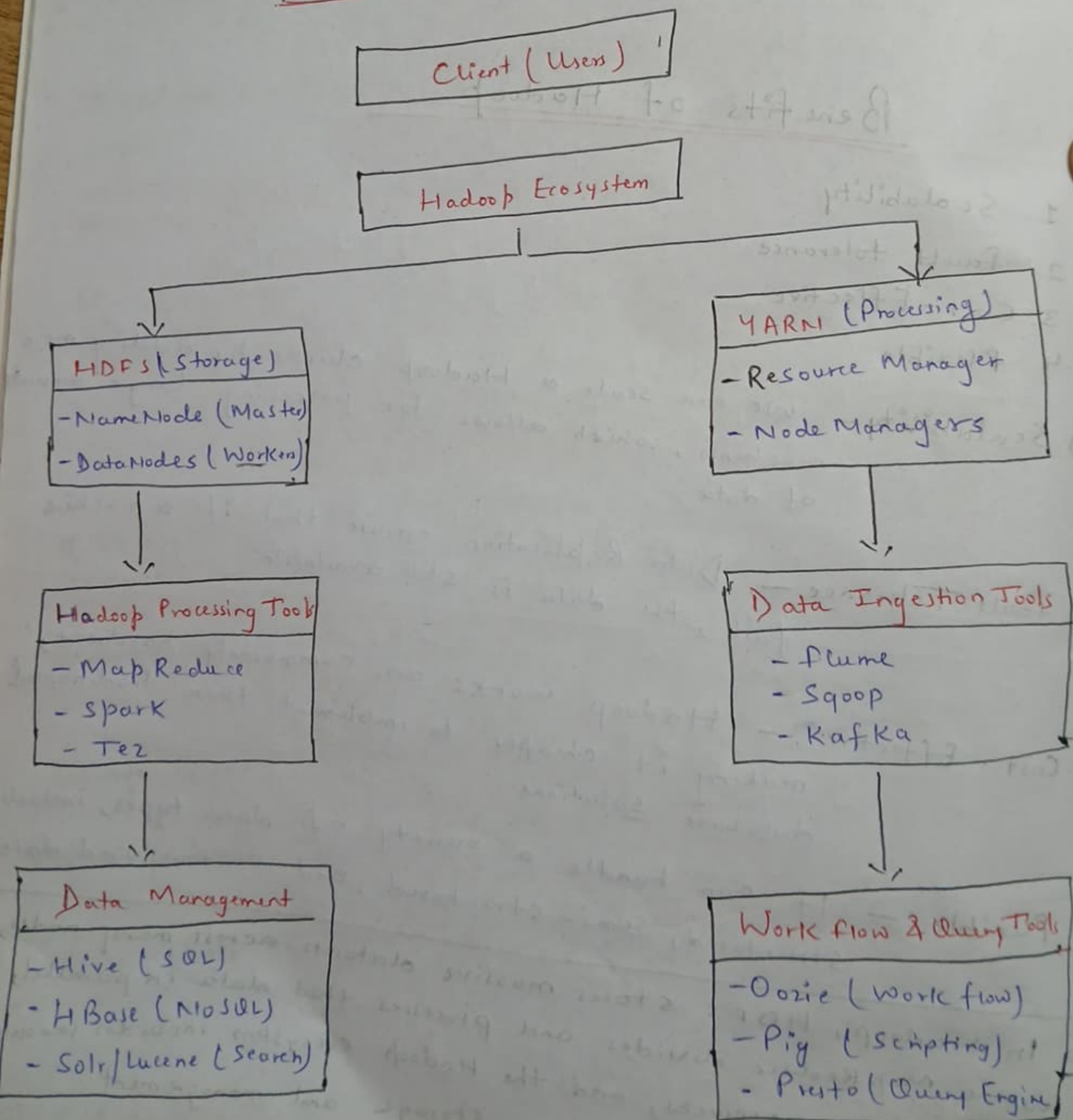
4.) Flexible : - It can handle a variety of data types, including structured, semi-structured, and unstructured data.

→ In Hadoop, HDFS stores massive datasets across many nodes, while MapReduce divides and processes that data in parallel, YARN manage resources, and the Hadoop ecosystem includes various projects to help with data processing storage and management.

Hadoop Challenges:

1. Complexity: Require expertise in distributed computing
2. Latency: Not ideal for real-time processing (better handle by Spark)
3. Security: Native Hadoop lacks robust security features (Kerberos can be used)

Hadoop Architecture Diagram



Benefit of Agility / Flexibility



Schema-on-Write (RDBMS)

- Schema must be created before any data can be loaded.
- An explicit load operation has to take place which transforms data to DB internal structure.
- New columns must be added explicitly before new data for such columns can be loaded into the database.

- Read is Fast
- Standards / Governance

Schema-on-Read (Hadoop)

- Data is simply copied to the file store, no transformation is needed.
- A SerDe (Serializer / Deserializer) is applied during read time to extract the required columns (*late binding*)
- New data can start flowing anytime and will appear retroactively once the SerDe is updated to parse it.

- Load is Fast
- Flexibility / Agility

